

Exercise 1: Variables and pointers

Write a program that takes two integer inputs from the user, stores them in variables a and b, and then displays their values using pointers ptrA and ptrB.

Exercise 2: Pointer and array traversal

Create a function void printArray(int* arr, int size) that takes an integer array and its size as arguments. Implement the function to print all elements of the array using pointer arithmetic, not array indexing (arr[i]).

Exercise 3: Passing by reference with pointers

Create a function void swap(int* ptr1, int* ptr2) that swaps the values of two integers. Call the function in main using pointers to two variables and display the values before and after the swap.

Exercise 4: Basic structure and pointer

1. Define a struct called Point with integer members x and y.
2. Declare a Point variable and a pointer to a Point variable.
3. Store the address of the Point variable in the pointer.
4. Use the arrow operator (->) to set the x and y values through the pointer and then display them.

Exercise 5: Array of structures

1. Define a struct called Student with members for name (std::string) and grades (an array of 5 integers).
2. Create an array of three Student structures.
3. Populate the data for each student.
4. Write a function that takes the array and its size, and prints the names of all students with an average grade over 90.

Exercise 6: Dynamic array with pointers

1. Write a function int* createDynamicArray(int size) that takes an integer size from the user, allocates memory dynamically for an integer array of that size, and returns a pointer to it.

2. In main, call this function, fill the array with user input, and then print the elements using the returned pointer.
3. Remember to deallocate the memory using `delete[]`.

Exercise 7: Pointers to structures in an array

1. Define a struct called `Product` with members `id` (int) and `price` (double).
2. Create an array of `Product` structures.
3. Create a separate array of pointers to `Product`.
4. Initialize the pointer array to point to the elements in the `Product` array.
5. Iterate through the array of pointers to print the details of each product.

Exercise 8: Structures, pointers, and dynamic memory

Create a dynamic address book application.

1. Define a struct `Contact` with members for `name` (`std::string`), `address` (`std::string`), and `phone_number` (`std::string`).
2. Use a pointer to a dynamically allocated array of `Contact` structures to store entries.
3. Implement functions to:
 - **Add new entries:** Dynamically resize the array to add a new contact.
 - **Display all entries:** Iterate through the array and print the details for each contact.
 - **Search for a contact:** Search for a contact by name and display their details if found.
 - **Free memory:** A function to deallocate all dynamically allocated memory.

Exercise 9: Pointer to pointer to char

1. Declare a `char` variable.
2. Declare a pointer to a `char` and a pointer to a pointer to a `char`.
3. Assign the address of the `char` to the `char` pointer.

4. Assign the address of the char pointer to the pointer to a pointer to a char.
5. Print the value of the char using:
 - The char variable itself.
 - Dereferencing the char pointer (*ptr).
 - Dereferencing the pointer to a pointer to a char twice (**ptr2).

Exercise 10: Array of pointers to structures

1. Define a struct Employee with members id (int), name (std::string), and salary (double).
2. Create an array of 5 Employee structures.
3. Create an array of 5 pointers to Employee.
4. Initialize the array of pointers to point to the corresponding Employee objects.
5. Implement a function void displayEmployees(Employee** employees, int size) that takes the array of pointers and its size as arguments. Inside the function, iterate through the array of pointers and display the employee data using the -> operator.